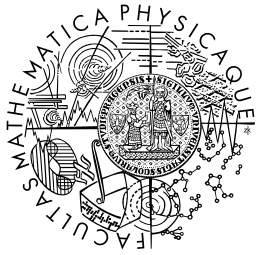

FACULTY
OF MATHEMATICS
AND PHYSICS
Charles University

MASTER THESIS

František Dostál

Fitness and novelty in evolutionary reinforcement learning

Name of the department

Supervisor of the master thesis: Mgr. Roman Neruda, CSc.
Study programme: Mgr.
Study branch: Artificial Inteligence

Prague 2023

I declare that I carried out this master thesis independently, and only with the cited sources, literature and other professional sources. It has not been used to obtain another or the same degree.

I understand that my work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In ..... date ....

Author's signature

Dedication.

Title: Fitness and novelty in evolutionary reinforcement learning

Author: František Dostál

Katedra teoretické informatiky a matematické logiky: Name of the department

Supervisor: Mgr. Roman Neruda, CSc., department

Abstract: Novelty is a novel aproach to modeling selection criteria in evolutionary algorithms and has been proven as viable technique of avoiding pitfalls of false optima in tasks abundant with them, such as solving mazes. Rather than closing the topic however, this finding opened other problems to explore: How to properly apply novelty in tasks that yield slightely better to conventional aproaches? How to properly model behavioral space nessessary for novelty computation? In this thesis we investigate use of novelty in selected reinforcement learnig tasks, it's combinations with classical fitness and propose behavior space models for the respective RL tasks.

Keywords: evolution novelty fitness behavioral space

Contents

Introduction **3**

1 Introduction to Artificial Agents **5**
1.1 Agents and environment 5
1.2 Neural Network Agent 6

2 Environments **7**
2.1 Reinforcement learning 7
2.2 Gymnasium 7
2.2.1 Cartpole 8
2.2.2 Lunar Lander 8

3 Evolutionary algorithms **9**
3.1 Continuous Optimisation 10
3.1.1 Evolutionary Strategies [Beyer and Schwefel, 2002] 10
3.1.2 Differential Evolution 11
3.2 Objective functions 11
3.2.1 Pure Novelty 11
3.2.2 Fitness 11
3.2.3 Combinations of Novelty 12

4 Comparison criteria and conditions **13**
4.1 General criteria 13
4.2 Conditions 13
4.3 Individual hyper-parameter selection 13
4.4 Algorithm hyperparameter selection 13
4.4.1 Bayesian Search 14
4.4.2 General Gridsearches 15
4.4.3 $\lambda + \mu$ hyperparameter selection 15
4.4.4 mutation sigma vs. generations 15
4.4.5 differential hyper-parameter selection 16

5 Experiments **17**
5.1 Hyperparameter selection 17
5.1.1 Bayesian Search 17
5.1.2 Iterative refinement 17
5.1.3 Generational grid search 17
5.2 Final comparison 17
5.2.1 Cartpole 17
5.2.2 Lunar Lander 17
5.3 Discussion 17

Conclusion **18**

Bibliography **19**

List of Figures **20**

List of Tables **21**

List of Abbreviations **22**

A Attachments **23**
A.1 First Attachment 23

Introduction

Evolutionary algorithms (EAs) are a class of optimization and search techniques inspired by the principles of natural evolution. Over the past decades, these algorithms have been successfully applied to a wide range of complex problems in engineering, robotics, and machine learning. Their ability to explore large and poorly understood search spaces makes them particularly suitable for problems where traditional optimization methods are applied only with great difficulty.

The performance of evolutionary algorithms is typically driven by a objective function, which evaluates candidate solutions according to predefined objectives. In classical evolutionary computation, this is usually refered to as fitness-based selection. While this approach has proven effective in many domains, it also presents several limitations. In particular, fitness-driven search can suffer from premature convergence, stagnation in local optima, and deception, where the search process becomes trapped in regions that appear promising but ultimately prevent discovery of globally optimal solutions.

To address these challenges, alternative search paradigms have been proposed, among which novelty search has gained significant attention. Unlike traditional fitness-based optimization, novelty search rewards behavioral uniqueness rather than objective performance. Introduced as a method to overcome deception in evolutionary search, novelty search encourages exploration by promoting individuals whose behaviors differ from those previously encountered. This shift from objective-oriented optimization to exploration-oriented search has demonstrated promising results in domains characterized by sparse rewards, deceptive landscapes, and high-dimensional complexity.

The distinction between fitness-based and novelty-driven approaches reflects two fundamentally different philosophies of evolutionary search. Fitness-oriented methods prioritize exploitation of known promising regions in the search space, whereas novelty-based methods emphasize exploration and behavioral diversity. Both approaches offer unique advantages and disadvantages depending on the characteristics of the problem domain. Fitness-based methods generally provide faster convergence toward measurable objectives but may lack robustness in deceptive environments. Novelty search, on the other hand, can discover innovative and unexpected solutions, although it may require greater computational resources and may struggle in tasks where exploration alone is insufficient.

This thesis focuses on a comparative analysis of novelty and fitness mechanisms in evolutionary algorithms when applied to reinforcement learning tasks. The objective is to investigate how these approaches influence search dynamics´, convergence behavior, diversity maintenance, and solution quality across selected optimization problems. The study aims to identify conditions under which novelty search outperforms traditional fitness-based evolution, as well as scenarios where fitness-driven optimization remains more effective.

The research combines theoretical analysis with experimental evaluation. Several benchmark problems and simulation environments are employed to compare the behavior of both methods under controlled conditions. Metrics such as convergence speed, population diversity, robustness, and final solution quality are analyzed to provide a comprehensive understanding of the strengths and limita-

tions of each approach. Additionally, hybrid methods that combine novelty and fitness objectives are considered to examine whether balancing exploration and exploitation can improve overall evolutionary performance.

The findings of this thesis contribute to the broader understanding of exploration and exploitation trade-offs in evolutionary computation. By examining the relationship between novelty and fitness, this work seeks to provide insights into the design of more adaptive and efficient evolutionary algorithms capable of solving increasingly complex optimization problems.

1. Introduction to Artificial Agents

To understand the reasoning guiding this work we have to first look at the fundamental subject of our efforts - the artificial agents.
We will look at their structure, guiding principles, practical implementation and nor last nor least, their use.

1.1 Agents and environment

All agents exists in environments that provide context to their agency. An agent gets information about the **environment** it find itself in (e.i. **percepts**) through **sensors** and acts on the environment through **actuators**. Both actuators and sensors are usually characteristics of the agent. Another characteristic of an agent would be setup of its internal machinations.

Definition 1. *[Russell and Norvig, 2021] Let E be environment where agent G is located. G is equipped with actuators A and sensors P and so we can identify $obss_P(E)$, a space of all possible sequences of percepts available to G within E and $A(E)$ space of available actions in E. Then function $G : obss_P(E) \to A(E)$ is called an agent function, defining behaviour of the agent G in environment E.*

We ilustrate relationship of these concepts in the diagram 1.1. Because we want our agents to solve some tasks in their respective enviroments with certain efficiency i.e. we want them to be intelligent, we need to define a notion of rationality that can help us better capture what we mean by intelligence. First however we need to measure how usefull or detrimental a situation can be for our agent.

Definition 2. *[Russell and Norvig, 2021] Be that E is an enviroment and $seq(E)$ is a space of all possible sequences of states within E. Then function $f : seq(E) \to \boldsymbol{R}$ is a performance measure of some agent operating in E.*

Definition 3. *[Russell and Norvig, 2021] For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.*

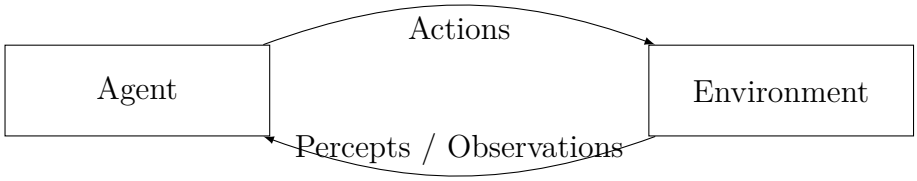

Figure 1.1: Agent and enviroment realtionship

We should note that in the real-world applications the rationality of an agents can often be limited by their capacity to store the evidence gathered from percepts, by the lack of their built-in knowledge or the computational capacity to estimate action sequence that can be expected to achieve global maximum of the agent's performance measure.
Since our focus is mainly on the evulationary algorithms in the scope of this thesis we will only encounter the most simple of agents subject to the most severe of such limitations.

Definition 4. *[Russell and Norvig, 2021] An agent is called simple reflex agent if it takes into account only the current percepts, never keeping any history or state information during it's run.*

1.2 Neural Network Agent

The concept of neural networks (NN) comes from an area of machine learning called deep learning that was concieved to attempt to model biological neuron activity. Since it has become one of the most successful branches of machine learning. [Russell and Norvig, 2021] The neural networks consist of neurons organised in layers.

Definition 5. *Let $W \in R^{mxn}$ and $b \in R^n$ and let $f : R^n \to R$.Then we call W the weights of a neuron layer with n neurons, b the bias of the layer and f the activation function of the layer when given an input vector $x \in R^m$ we calculate the output of the layer as $f(W^T x + b)$. Two layers have directional connection when the output of first layer is used as input of the subsequent layer.*

The purpose of the activation function is to introduce exploitable non-linearity into the neural system. We recognise several basic activation function.

- sigmoid
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$
- tanh
$$\tanh(x) = \frac{e^{2x} - 1}{e^{2x} + 1}$$
- ReLU
$$ReLU(x) = max(0, x)$$

Definition 6. *[Russell and Norvig, 2021] A neural network where the connections between layers of neural network form an acyclic graph is called a feed-forward neural network.*

Because the feed-forward NN doesn't hold any state or holds onto any previously acquired results, only produces output for a given input, they are ideal for implementation of simple reflex agents. In this thesis we use feed-forward netwroks exclusively, so any further mention of neural netwrok should be assumed to be feed-forward.

2. Environments

2.1 Reinforcement learning

Reinforcement learning might be best introduced by description of the problem it aims to solve.

Definition 7 (Markov's decision problem)**.** *We define Markov's decision problem (MDP) by a four-tuple (S, A, P, R) where:*

- *S is a set of all possible states of the system*
- *A is a set of all possible actions available in the system*
- *P is a transition function,$P(s, a, s')$ being probability of transition from s to s' give action a is executed.*
- *R is reward function, $R(s, a, s')$ being the reward received when the execution of action a in state s leads to state s'.*

To tie this into the information from chapter 1 $A = A(E)$, R is performance measure of an agent solving the MDP and the percepts correspond to real states of the environment. This would not be the case for partially observable MDP.

Definition 8. *POMDP*

Definition 9. *Best policy*

The goal of reinforcement learning is then to proximate best policy by trial and error, based on the received reward, as accurately as possible. This can be done utilising various methods such as Q-learning, but importantly for this thesis it could be

2.2 Gymnasium

In this thesis we will set our agents into environments provided by Gymnasium project which emerged from it's predecessor OpenAI Gym project. Both of them Python libraries, trying to provide researches with benchmark reinforcement learning problems in form of simple games, sometimes even recreating old console games such as Air Raid or Backgammon.

While Gymnasium provides wide range of environments, quite a number of them if not most do not provide their outputs in structured form. Instead the resulting screen has to be processed by user defined visual processor to extract information crucial for completing the task.

This introduces a layer of complexity that is outside of the scope of this thesis. Therefore we have decided to narrow our selection only to environments which do provide structured output. This is the two environments we have eventually selected for our study:

2.2.1 Cartpole

Cartpole is one of the most used benchmark environments in the Gymnasium library. It's a simple environment where the agent controls a cart rolling on flat surface with a pole on top. The goal is to keep the pole balanced on top of the cart. The reward is calculated every step depending on the angle of the stick. This means we have bounded

Definition 10. *An agent that achieves score 500 in Cartpole is called solution or solving individual.*

Observation space

Action space

2.2.2 Lunar Lander

The other environment we have selected is Lunar Lander where the agent controls descend of a landing shuttle on rugged planetary (or the Moon's) surface with the help of strangely placed rockets. The points are subtracted for staying in the air or for getting the shuttle into a spin. Bonus points awarded for landing in the center of the The shuttle has infinite fuel so the game ends only when the shuttle lands safely or crushes into the ground. Luckily we can at least limit the number of steps for which the environment is allowed to go on. For a neural network training this however poses a specific kind of problem. Firstly very narrow behavioral window that can achieve positive fitness. But mainly as opposed to the Cartpole environment (2.2.1), now the unsuccesful runs may take significantly longer time than the successful ones. This means experiments are particularly costly for slowly converging algorithms. The upper limit of fitness is 200, the lower would possibly be $-\inf$, depending on the lenght limitation imposed. We therefore differntiate between two stages of obtaining successful solution.

Definition 11. *An agent that achieves score 200 in Lunar lander is called solution or solving individual.*

Definition 12. *An agent that achieves positive score in Lunar lander is called well-behaved.*

Definition 12 stems from the fact that it can only be achieved if the agent does not crash the shuttle as doing so yields a penalisation of -100 points while not obtaining a boost of +100 points for safe landing, driving even a possible solution with score 200 to 0.

Observation space

Action space

3. Evolutionary algorithms

The evolutionary algorithms (EA) are a class of optimisation methods modelled after our ideas of biological process of evolution.

Definition 13. *General optimisation problem takes the form:*

$$\begin{aligned} &\textit{minimise:} && f(x) \\ &\textit{subject to:} && g_i(x) \leq 0, \quad i = 1, \ldots, m, \\ & && h_j(x) = 0, \quad j = 1, \ldots, p, \end{aligned}$$

where is the solution vector, f is objective function,
and g_i, h_j are constraints imposed on x

Often in literature on EA we might find confluence of the term objective function and another term borrowed from biology - fitness. For reasons that will become apparent in the course of this chapter we will make sharp distinction between them for he purposes of this thesis. By objective function we will understand function for which the algorithm directly optimises while fitness will be used to describe semantic quality of an individual in the context of its use. Because of this biological connection solution vectors are in the literature often referred to as individuals and we will refer to them this way henceforth as well. Alternatively when emphasising either the numerical form of the vector or semantic interpretation of this form beyond the optimisation algorithm the individual may be refer to as either a genotype or phenotype respectively.
Group of individuals representing candidate solutions at a given time is referred to as a population. Evolutionary algorithms then evolve the population by applying three operations (operators) to it in a loop until either the population converges to optimum or the limit on number of steps is reached.

Algorithm 1: Generalised Evolutionary Algortihm

```
Data: Initial population = pop
Data: N = Number of generations
Result: Optimised population = pop
for g ∈ 1..N do
    co ← CrossOver(pop);
    m ← Mutate(co);
    pop ← Select(m);
end
return pop
```

Crossover usually involves recombination of features from two or more individuals from the population. It's purpose is to provide means of exploitation of features beneficial to the population already developed by some individual.
On the other hand mutation provides means of exploration by introducing randomness into the individuals.
EA without crossover would be more akin to a random walk algorithm. Selection

then is about filtering the resulting individuals based on the objective function of the algorithm. It might be performed deterministically or with varying degree of stochasticity as elaborated in section 3.1.

3.1 Continuous Optimisation

Historically the individuals optimised by EA used be primarily concieved as vectors over binary domain - $\{0, 1\}$. However this aproach was due to practical concerns later expanded into the domain of vectors over real numbers - in litarature refered to as continuous optimisation. For this purpose the operators of EAs had to be adapted to the domain. The literature shows many aproaches without any obvious limit to their variety. In this thesis we have will explore only two most widespread evolutionary aproaches to continuous optimisation.

3.1.1 Evolutionary Strategies [Beyer and Schwefel, 2002]

Evolutionary strategies (ES) are very direct incarnation of algorithm 1 into the realm of real numbers. Distinct feature of evolutionary staregies is the mutation operator that takes as a parameter standard devaition σ and which mutates the value stochastically by a number y $N(0, \sigma)$ We recognise two types of evolutionary strategies: $\lambda + \mu$ and λ, μ.

Algorithm 2: Evolutionary Strategy $\lambda + \mu$

Data: population count, offspring count, number of generations
Result: optimised population
$\lambda \leftarrow$ population count;
$\mu \leftarrow$ offspring count ;
$pop \leftarrow InitPop(\lambda)$;
for $g \in 1..N$ **do**
  $off \leftarrow CrossOver(pop, \mu)$;
  $off \leftarrow Mutate(off)$;
  $pop \leftarrow Select(pop + off, \lambda)$;
end
return *pop*

Algorithm 3: Evolutionary Strategy λ, μ

Data: population count, offspring count, number of generations
Result: optimised population
$\lambda \leftarrow$ population count;
$\mu \leftarrow$ offspring count ;
$pop \leftarrow InitPop(\lambda)$;
for $g \in 1..N$ **do**
  $off \leftarrow CrossOver(pop, \mu)$;
  $off \leftarrow Mutate(off)$;
  $pop \leftarrow Select(off, \lambda)$;
end
return *pop*

3.1.2 Differential Evolution

While evolutionary strategies follow quite closely the general blueprint of EAs across domains, differential evolution is on the hand quite unique for the domain of continuos optimisation. While the nomenclature of differential evolution is somewhat different from other EA we ask the reader to be mindful of analogy between recombination and mutation when we talk about evolutionary algorithms broadly while we will use the proper terms when talking about the differential evolution specifically.

3.2 Objective functions

As foreshadowed at the start of this chapter it might be desirable to divorce the utility (fitness) of an individual from the objective the algorithm follows in it's search. In this thesis we are interested in objective function of novelty **?** and it's combinations with fitness.

3.2.1 Pure Novelty

In its introductory paper novelty **?** is conceived as a guide of the evolutionary search to help avoid traps of local optima.

Definition 14. *[Doncieux et al., 2019] Let E be environment and S be a space of all possible states of the environment and let $d(x, y)$ be some metric. Then (B, d) is a behavioural space and a function $o_B : S^T \to B$ is called observer function.*

Definition 15. *[?] Let P be a population of individuals within EA and (Img(B), d) be a behavioural space. Then for each individual a we define novelty as such:*

$$N(a) = \frac{\sum_{b \in P, b \neq a} d(a, b)}{|P| - 1}$$

With novelty as objective function the selection process incentivises diversification of behaviour among the population. This allows the evolutionary search to retain variety within the population across generations, leveraging the crossover as well as mutations to leave local optima.
Downside of this approach might be in the ambiguity in the choice of behavioural space. Incorporating too many environment variables will lead to faux-exploration in variables completely orthogonal to fitness. On the other hand leaving out an important variable means the algorithm will not retain diversity where it matters, making novelty pointless. This means the behavioural function and space have to be selected empirically for given problem.

3.2.2 Fitness

Using objective function as the utility function of the algorithm is the most direct way of establishing relationship between the solution we want and the individuals in the evolving population.
Though this approach ensures that the most optimal individuals from given population will generally be selected into the next generation during the evolution it

does not guarantee that their potential for evolving is not diminished either. This way the evolution using fitness can easily get stuck in local optimum of the objective function. While in most cases it might be still possible to reach global optimum on the given problem by changing the hyper-parameters to favour random exploration, it may come to point where the benefits of evolutionary approach are made void.

3.2.3 Combinations of Novelty

The novelty does not take into account the individual's performance with objective function. Therefore when two individuals with similar behavioural characteristics appear but one of them performs significantly better than the other, pure novelty algorithm can easily select the less performant individual, losing the more performant genome for further generations. To prevent this we want to meaningfully combine novelty with fitness.
One obvious way of doing this is

4. Comparison criteria and conditions

To compare different types of objective functions fairly, we have to select criteria by which we compare them and establish conditions under which will this comparison take place so that our conclusions are fair.

4.1 General criteria

Since fitness provides direct measure of performance we would like to make our comparison on the fitness of the final population and its distribution in it. To that end we have to perform experiments with variety of initialisations of the respective environments, getting more comprehensive picture then from a single run. This will be mainly reflected in the seed for random numbers generator of the environment. Since the promise of the novelty approach lies mainly in avoiding the trap of local optima we also want to review statistics of each generation produced by the algorithm runs
However we also have to take into account the hyper-parameters with which these populations have been achieved. This is especially (but not limited to) population size and number of generations, hyper-parameters responsible for most of the computational complexity of a evolutionary algorithm run.

4.2 Conditions

The conditions are defined by the hyper-parameters of the algorithms and the hyper-parameters of the individuals.

4.3 Individual hyper-parameter selection

The hyper-parameters characterising individuals are number of layers and the number of neurons in their layers. The choice of these parameters is specific for each task. Generally the more complex is the task the more of either layers or neurons per layer the handling NN requires to be suitable for it. We determined these parameter based on empirical experience in the chapter 2.

4.4 Algorithm hyperparameter selection

We have reviewed hyperparamets of each algorithm in the chapter 3. Now we will review the process of their selection. The diffulty of the task lies mainly in the complexity of evaluation. As we will discuss later some of the experiments maybe evaluated over day or even several days on the available hardware even for a 100 limited algorithm runs. For this reason we need to utilise bayesian search which is able to prune a lot of the redundant space out so that we may not waste time with it. This however has a considerable downside for our purposes as we

would like to compare the algorithms on their best footing. Therefore we refine the bayesian search results with local grid search attempting to deviate from the supposed hyperparameter equilibria.

4.4.1 Bayesian Search

Because we are faced with multiple Bayesian search is an algorithm where we stochastically deliniate where a possibly good solution may lie and then try to exploit this deliniation. Because the specific workings of Bayesian search are time-consuming to implement ad hoc (and also such implementation is out of the scope of this thesis) I have utlised library Optuna for the purpose.

Optuna

Just as in our case a lot of machine-learning techniques need for their proper function to obtain adequate hyperparameters first. Optuna is a Python library designed for this purpose. The library itself is multifasceted, so we will only discuss algortihms and terms pertaining to the subject of this thesis. ¡define trial¿ ¡define study¿ ¡define optimisiation run¿

As the optimisation algorithm we have selected Tree-structured Parzen Estimator (TPE) due to the fairly low complexity and efectivness, wasting no trials as we want to conserve as much actual runtime as possible.

TPE

The Tree-structured Parzen Estimator (TPE) used in Optuna is a sequential model-based optimization method for hyperparameter tuning that reformulates Bayesian optimization in terms of density estimation over the input space rather than the objective space. It was introduced by Bergstra et al. (2011) as part of early work on scalable hyperparameter optimization Bergstra et al. [2011].

In a standard hyperparameter optimization setting, we observe a dataset of trials

$$\mathcal{D} = \{(x_i, y_i)\},$$

where x_i is a hyperparameter configuration and y_i is the corresponding loss (or score). TPE begins by splitting these observations into two groups using a threshold y^*, typically chosen as a quantile of observed outcomes:

- **Good configurations:** those with $y_i < y^*$
- **Bad configurations:** those with $y_i \geq y^*$

This split is not arbitrary—it defines what the algorithm currently considers promising based on past experience.

Instead of learning a direct predictive model $p(y \mid x)$, TPE learns two simpler distributions over configurations:

$$\ell(x) = p(x \mid y < y^*), \quad g(x) = p(x \mid y \geq y^*),$$

These distributions are typically estimated using smooth non-parametric density estimators (Parzen estimators), which can flexibly model continuous, discrete,

and categorical hyperparameters without strong assumptions about the shape of the space. The key idea is that the next configuration should be one that is:

- likely under the "good" distribution, and
- unlikely under the "bad" distribution.

This leads to the selection rule:

$$x^* = \arg\max_x \frac{\ell(x)}{g(x)}.$$

Rather than optimizing this expression directly, TPE samples candidate points from $\ell(x)$ and evaluates them using this ratio, selecting the best candidate. This makes the method computationally efficient and avoids difficult global optimization of an acquisition function.

4.4.2 General Gridsearches

Now we will discuss grid searches that are common for all implementations of our EAs. Note that for convenience we do not present these concepts in the temporal order in which they have been evaluated.

evaluation episodes vs. generations

Before we start other experiments we have to determine how many episodes will be used to evaluate an individual during the algorithm's run in these experiments. The parameter directly influences how much information the algorithm has extracted about the individual, as more episodes will lead to more indicative behavioural descriptors. However since the episodes take up non-trivial part of the computation we want to select the lowest possible number. For this purpose we will run a grid search comparingdifferential algorithm's performance over different number of generations. Since the Gymnasium environments discussed in chapter 2. are based on similair principles we can be confident about generation of this result to conserve time for more meaningful experiment. ¡insert parameters¿

generations grid search

After obtaining refined solution from each EAs respective incremental grid search

4.4.3 Incremental grid search

Because we obtained the optimised parameters from Bayesian search stochastically we would like to ensure that there is no easily obtainable set of parameters more performant than the set from BS. All however in time consuming fashion. To this end we have devised an algorithm of incremental grid search where we evaluate possibly useful deviations from the supposed local optima. Upon dicovering new optima we recenter the search around it. If we are not successfull in finding dominating set of parameters we end the search with the last obtained optima. We also end the search after more than 3 recentering. Since this is only refining algorithm of already functional solution we do not expect long searches.

4.4.4 $\lambda + \mu$ incemental grid search

4.4.5 DEs incremental grid search

5. Experiments

5.1 Hyperparameter selection

5.1.1 Bayesian Search

5.1.2 Iterative refinement

5.1.3 Generational grid search

5.2 Final comparison

5.2.1 Cartpole

5.2.2 Lunar Lander

5.3 Discussion

Conclusion

I have successfully developed and implemented framework for testing evolutionary strategies and differential evolution in several setups, such as fitness, novelty and their respective archivng and a combination of them. I have also proposed behavioral models for both environments Lunar Lander and Cartpole. These behavioral models are a modest success, while Lunar Lander model, unlike Cartpole, does not consistently achieve solution grade score, it is still very well-behaved. I have also proposed a hyperparameter searching algorithm based on Bayesian search. While the search itself is rather limited, this reflects the conditions of the used environments, which are time-exhausting to simulate in large numbers. The algorithm allows us to compare the EAs on their performance given hyperparameters found by this search. The areas of comparisons were: absolute performance, rate of convergance and diversity of the final populations.

Bibliography

James S. Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2011.

Hans-Georg Beyer and Hans-Paul Schwefel. Evolution strategies - a comprehensive introduction. *Natural Computing*, 1:3–52, 03 2002. doi: 10.1023/A:1015059928466.

Stephane Doncieux, Alban Laflaquière, and Alexandre Coninx. Novelty search: a Theoretical Perspective. In *GECCO '19: Genetic and Evolutionary Computation Conference*, pages 99–106, Prague Czech Republic, France, July 2019. ACM. doi: 10.1145/3321707.3321752. URL `https://hal.science/hal-02561846`.

Stuart Russell and Peter Norvig. *Artificial Intelligence, Global Edition A Modern Approach*. Pearson Deutschland, 2021. ISBN 9781292401133. URL `https://elibrary.pearson.de/book/99.150005/9781292401171`.

List of Figures

1.1 Agent and enviroment realtionship 5

List of Tables

List of Abbreviations

A. Attachments

A.1 First Attachment